

CS683 : Programming Assignment 1

Group Name: Cache Me If You Can

Group Members

Sai Thenge	24B1206
Khushi Kucheria	24B1219
Hamshini Kanuri	24B1253
Soham Dusane	24B3924

Contents

1	Task 1	1
1.1	Part A	1
1.2	Part B	3
1.3	Part C	7
1.4	Part D	10

1 Task 1

1.1 Part A

In this part, we were asked to optimize a naive 2D convolution implementation in two ways:

1: Loop Unrolling

We stepped the `ox` loop by 4 instead of 1, and inside each iteration we maintained 4 separate accumulators (`acc0-acc3`), one for each of the 4 output columns being computed together. For every kernel weight, we updated all 4 accumulators in the same inner-loop pass, and finally wrote all 4 results to the output array at once. This reduces the number of times the loop counter is incremented and bounds are checked, since the `ox` loop now runs $W/4$ times instead of W times.

Listing 1: Loop Unrolling

```
1 void conv_unroll(const float* in, float* out, const float* ker,
2                 int H, int W, int K) {
3     const int p = K / 2;
4     const int in_stride = W + 2 * p;    // padded row stride
5
6     for (int oy = 0; oy < H; ++oy) {
7         for (int ox = 0; ox < W; ox += 4) {
8             float acc0 = 0.0f, acc1 = 0.0f, acc2 = 0.0f, acc3 =
9                 0.0f;
10
11             for (int ky = 0; ky < K; ++ky) {
12                 for (int kx = 0; kx < K; ++kx) {
13                     float w = ker[ky * K + kx];
14                     acc0 += w * in[(oy + ky) * in_stride + ox +
15                                   kx + 0];
16                     acc1 += w * in[(oy + ky) * in_stride + ox +
17                                   kx + 1];
18                     acc2 += w * in[(oy + ky) * in_stride + ox +
19                                   kx + 2];
20                     acc3 += w * in[(oy + ky) * in_stride + ox +
21                                   kx + 3];
22                 }
23             }
24
25             out[oy * W + ox + 0] = acc0;
26             out[oy * W + ox + 1] = acc1;
27             out[oy * W + ox + 2] = acc2;
28             out[oy * W + ox + 3] = acc3;
29         }
30     }
```

2: Loop Reordering

In the naive convolution, the kernel loops (**ky**, **kx**) are nested inside the output loops (**oy**, **ox**). This means every time we move to a new output cell, the kernel is accessed all over again from the start.

So we first initialized the entire output array to zero. Then, for each kernel element `ker[ky][kx]`, we looped over all output cells (**oy**, **ox**) and added its contribution into `out[oy][ox]`. This way, a given kernel value is loaded once and reused across the entire output computation for that value, instead of being re-fetched for every output pixel.

Listing 2: Loop Reordering

```
1 void conv_reorder(const float* in, float* out, const float* ker,
2                   int H, int W, int K) {
3     const int p = K / 2;
4     const int in_stride = W + 2 * p;    // padded row stride
5
6     for (int oy = 0; oy < H; ++oy)
7         for (int ox = 0; ox < W; ++ox)
8             out[oy * W + ox] = 0.0f;
9
10    for (int ky = 0; ky < K; ++ky) {
11        for (int kx = 0; kx < K; ++kx) {
12            int conv_vl = ker[ky * K + kx];
13            for (int oy = 0; oy < H; ++oy) {
14                for (int ox = 0; ox < W; ++ox) {
15                    out[oy * W + ox] +=
16                        in[(oy + ky) * in_stride + (ox + kx)] *
17                        conv_vl;
18                }
19            }
20        }
21    }
```

What motivated us to make changes?

I think the most important reason behind our motivation to make changes to the given code was umm because the problem statement asked us to :)

What considerations did we take in account?

- that the output should still be the same. like the unrolling and reordering should hamper our outputs
- for unrolling we assumed the W to be perfectly divisible by 4
- obviously that W is greater than 4 or else the unrolling would give bound errors hehe

Effectiveness of our changes

So we would use the time and speedup to see how effective my changes were.

```
=== Workload H=2048 W=2048 K=3 seed=1234 (not graded) ===
stage        correct    time(ms)    GFLOP/s    speedup
-----
naive (ref)   yes             41.939       1.80      1.00x
reorder       yes             24.612       3.07      1.70x
CS683 PA-1 Task 1  2D convolution optimization harness

=== Workload H=2048 W=2048 K=3 seed=1234 (not graded) ===
stage        correct    time(ms)    GFLOP/s    speedup
-----
naive (ref)   yes             22.848       3.30      1.00x
unroll        yes             20.024       3.77      1.14x
```

```
=== Workload H=2048 W=2048 K=5 seed=1234 (not graded) ===
stage        correct    time(ms)    GFLOP/s    speedup
-----
naive (ref)   yes             69.153       3.03      1.00x
reorder       yes             71.771       2.92      0.96x
CS683 PA-1 Task 1  2D convolution optimization harness

=== Workload H=2048 W=2048 K=5 seed=1234 (not graded) ===
stage        correct    time(ms)    GFLOP/s    speedup
-----
naive (ref)   yes             86.557       2.42      1.00x
unroll        yes             95.853       2.19      0.90x
```

```
=== Workload H=2048 W=2048 K=7 seed=1234 (not graded) ===
stage        correct    time(ms)    GFLOP/s    speedup
-----
naive (ref)   yes            168.787       2.44      1.00x
reorder       yes            154.580       2.66      1.09x
CS683 PA-1 Task 1  2D convolution optimization harness

=== Workload H=2048 W=2048 K=7 seed=1234 (not graded) ===
stage        correct    time(ms)    GFLOP/s    speedup
-----
naive (ref)   yes            197.753       2.08      1.00x
unroll        yes            134.865       3.05      1.47x
```

1.2 Part B

In this part, we were asked to further optimize the convolution using **cache tiling** (also called loop blocking).

Cache Tiling

Instead of sweeping across the entire output row by row (which can cause the same cache lines to be evicted and reloaded repeatedly for large images), we divided the output matrix into smaller $\text{TILE} \times \text{TILE}$ blocks. In tiling, we process one block completely before moving to the next. Since a whole block (and the input region it depends on) fits inside the cache, the data stays resident while the kernel loop runs over it, so by the

time we move to the next row within the same tile, those cache lines are still present and we get cache hits instead of misses. The tile size may not evenly divide H and W, so we clamped the tile boundaries using `std::min(ty + TILE, H)` and `std::min(tx + TILE, W)` to avoid going out of bounds on the last (partial) tile in each dimension.

1. Profiling baseline 2D convolution code:

```
L1d cache: 192 KiB (4 instances)
48K
1
Data
LEVEL1_ICACHE_SIZE 32768
LEVEL1_ICACHE_ASSOC
LEVEL1_ICACHE_LINESIZE 64
LEVEL1_DCACHE_SIZE 49152
LEVEL1_DCACHE_ASSOC 12
LEVEL1_DCACHE_LINESIZE 64
LEVEL2_CACHE_SIZE 1310720
LEVEL2_CACHE_ASSOC 20
LEVEL2_CACHE_LINESIZE 64
LEVEL3_CACHE_SIZE 8388608
LEVEL3_CACHE_ASSOC 8
LEVEL3_CACHE_LINESIZE 64
LEVEL4_CACHE_SIZE
LEVEL4_CACHE_ASSOC
LEVEL4_CACHE_LINESIZE
```

perf outputs for naive model

$$MPKI = 1.765 \quad (1)$$

```
Performance counter stats for './bin/conv naive 2048 2048 3' (10 runs):

 4449285699      instructions:u      ( +- 0.00% )
 868817314      L1-dcache-loads:u      ( +- 0.00% )
 7856295        L1-dcache-load-misses:u      ( +- 1.34% )

0.336113103 +- 0.015848789 seconds time elapsed ( +- 4.72% )
```

2. Implementing Tiled 2D convolution

Listing 3: Cache Tiling

```
1 #ifndef TILE
2 #define TILE 32
3 #endif
4
5 void conv_tile(const float* in, float* out, const float* ker,
6               int H, int W, int K) {
7     const int p = K / 2;
8     const int in_stride = W + 2 * p;
9
10    for (int ty = 0; ty < H; ty += TILE) {
11        int oy_end = std::min(ty + TILE, H);
12        for (int tx = 0; tx < W; tx += TILE) {
13            int ox_end = std::min(tx + TILE, W);
```

```

14
15     for (int oy = ty; oy < oy_end; ++oy) {
16         for (int ox = tx; ox < ox_end; ++ox) {
17             float acc = 0.0f;
18             for (int ky = 0; ky < K; ++ky) {
19                 for (int kx = 0; kx < K; ++kx) {
20                     acc += in[(oy + ky) * in_stride + (ox
21                             + kx)]
22                             * ker[ky * K + kx];
23                 }
24             }
25             out[oy * W + ox] = acc;
26         }
27     }
28 }
29

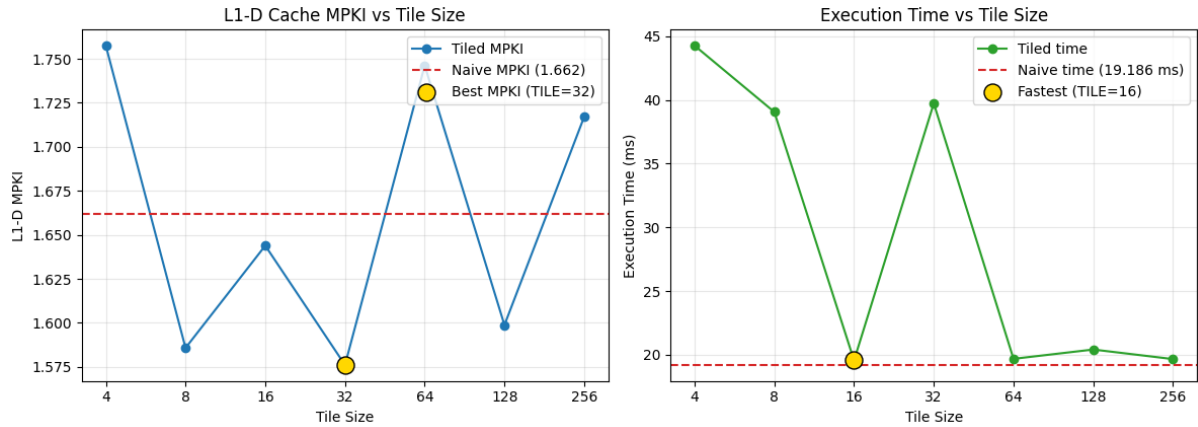
```

Experimenting with various tile sizes

2	1024	89.094	1.01x	2.922
2	2048	86.030	1.05x	2.925
2	4096	78.473	1.15x	2.923
4	32	99.629	.90x	2.920
4	64	81.314	1.11x	2.916
4	128	84.864	1.06x	2.934
4	256	97.083	.93x	2.919
4	512	112.221	.80x	2.925
4	1024	100.529	.90x	2.932
4	2048	106.245	.85x	2.916
4	4096	136.523	.66x	2.917
8	32	81.328	1.11x	2.921
8	64	86.369	1.04x	2.941
8	128	88.141	1.02x	2.927
8	256	82.811	1.09x	2.925
8	512	113.284	.79x	2.926
8	1024	95.835	.94x	2.940
8	2048	80.953	1.11x	2.926
8	4096	92.631	.97x	2.939
16	32	103.950	.87x	2.935
16	64	91.477	.99x	2.933
16	128	93.066	.97x	2.920
16	256	114.163	.79x	2.932
16	512	104.015	.87x	2.923
16	1024	86.433	1.04x	2.926
16	2048	87.639	1.03x	2.921
16	4096	105.415	.85x	2.912

Rectangular tile sizes didn't give good results, we wrote the code and plotted the graph for only square tile sizes.

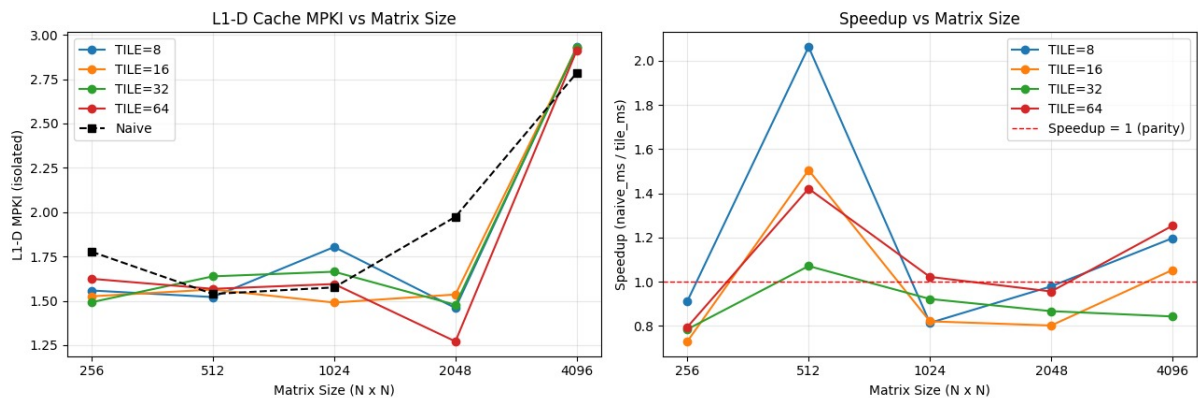
Tiled 2D Convolution: Cache Behavior vs Tile Size (H=W=2048, K=3)



Experimenting with various matrix sizes

```
size, tile, naive_ms, tile_ms, speedup, naive_MPKI, tile_MPKI_isolated
256, 8, 0.384, 0.421, .912, 1.7770, 1.5584
256, 16, 0.384, 0.526, .730, 1.7770, 1.5256
256, 32, 0.384, 0.489, .785, 1.7770, 1.4924
256, 64, 0.384, 0.483, .795, 1.7770, 1.6243
512, 8, 3.584, 1.738, 2.062, 1.5390, 1.5214
512, 16, 3.584, 2.380, 1.505, 1.5390, 1.5640
512, 32, 3.584, 3.346, 1.071, 1.5390, 1.6382
512, 64, 3.584, 2.521, 1.421, 1.5390, 1.5677
1024, 8, 4.760, 5.847, .814, 1.5758, 1.8036
1024, 16, 4.760, 5.792, .821, 1.5758, 1.4904
1024, 32, 4.760, 5.161, .922, 1.5758, 1.6643
1024, 64, 4.760, 4.657, 1.022, 1.5758, 1.5949
2048, 8, 21.410, 21.856, .979, 1.9738, 1.4598
2048, 16, 21.410, 26.680, .802, 1.9738, 1.5358
2048, 32, 21.410, 24.674, .867, 1.9738, 1.4777
2048, 64, 21.410, 22.389, .956, 1.9738, 1.2712
4096, 8, 106.164, 88.664, 1.197, 2.7848, 2.9319
4096, 16, 106.164, 100.786, 1.053, 2.7848, 2.9267
4096, 32, 106.164, 125.915, .843, 2.7848, 2.9259
4096, 64, 106.164, 84.727, 1.253, 2.7848, 2.9115
```

Tiled 2D Convolution Performance vs Matrix Size (K=3)



ANSWERING QUESTIONS

1. we went from naive to tiled convolution, the MPKI didn't go down uniformly, it depended on the matrix size. For small and medium matrices (256 to 2048), tiling generally reduced MPKI, in some cases by over 30%. This makes sense because tiling breaks the big matrix into small parts that fit better inside the L1 cache, so the CPU doesn't have to keep reloading data from slower memory.

But for the largest matrix (4096), tiling actually made MPKI slightly worse. At that size, even a small tile plus its border data (needed for the convolution overlap) was too much for the cache to hold efficiently, so the benefit of tiling was lost.

2. As the matrix size increased, MPKI generally went up too, because bigger matrices don't fit in the small L1 cache, forcing more data to be reloaded from slower memory, a classic capacity problem.

Tile size also mattered. For small matrices, small tiles worked best because they were light and simple. For large matrices, bigger tiles worked better because they reduced the extra overhead of managing many tiny tiles. In between, MPKI actually went up and down as tile size changed. We think this happened because certain tile sizes accidentally cause the cache to overwrite useful data (cache conflicts), even though there's technically enough room.

3. Yes, but only sometimes. Tiling gave a speedup (up to about 2x, for medium-sized matrices with small tile sizes), and a smaller increase for the largest matrix with bigger tiles. But for several other combinations, tiling was actually slower than the simple naive version. This happened because tiling adds extra overhead (managing tile boundaries and loops), which isn't worth it if the matrix is small or the tile size doesn't suit the cache well. A better approach would be to choose the best tile size for each matrix size instead of using one fixed value.

1.3 Part C

In this part, we were asked to optimize the convolution using **SIMD (Single Instruction, Multiple Data)** vectorization, so that multiple output values are computed in parallel within a single instruction.

SIMD Vectorization

We used 256-bit AVX registers (`_mm256`), each of which holds 8 `float` values. so instead of computing one output at a time, we compute 8 outputs together in a single vector operation.

For each kernel weight, we convert it into a vector (so it's replicated across all 8 lanes) and multiply-accumulate it with 8 consecutive input values at once using `_mm256_fmadd_ps` (fused multiply-add), which performs the multiplication and addition in a single instruction.

Since the kernel values don't change across the whole image, we precompute all the

broadcasted kernel weights once per call (cached in `ker_bcast`) instead of redoing it for every pixel.

Since W may not be a multiple of 8, we process the bulk of each row in chunks of 8 using SIMD, and handle the remaining leftover pixels (fewer than 8) at the end of the row using a regular scalar loop, identical to the naive version.

1. Baseline Profiling:

$$no.of\ instructions = 4449285699 \quad (2)$$

```
Performance counter stats for './bin/conv naive 2048 2048 3' (10 runs):

    4449285699      instructions:u                ( +- 0.00% )
    868817314      L1-dcache-loads:u            ( +- 0.00% )
    7856295        L1-dcache-load-misses:u       ( +- 1.34% )

0.336113103 +- 0.015848789 seconds time elapsed ( +- 4.72% )
```

2. SIMD Implementation:

modified and ran the code for all 128, 256 and 512 bit registers

Listing 4: SIMD Vectorization

```
1 void conv_simd(const float* in, float* out, const float* ker,
2               int H, int W, int K) {
3     const int p = K / 2;
4     const int in_stride = W + 2 * p;
5
6     static thread_local std::vector<__m256> ker_bcast;
7     ker_bcast.resize(K * K);
8     for (int i = 0; i < K * K; ++i) ker_bcast[i] = _mm256_set1_ps
9         (ker[i]);
10
11     for (int oy = 0; oy < H; ++oy) {
12         int ox = 0;
13
14         for (; ox + 8 <= W; ox += 8) {
15             __m256 acc = _mm256_setzero_ps();
16             for (int ky = 0; ky < K; ++ky) {
17                 for (int kx = 0; kx < K; ++kx) {
18                     const float* row_ptr =
19                         &in[(oy + ky) * in_stride + (ox + kx)];
20                     __m256 in_vec = _mm256_loadu_ps(row_ptr);
21                     acc = _mm256_fmadd_ps(in_vec, ker_bcast[ky *
22                         K + kx], acc);
23                 }
24             }
25             _mm256_storeu_ps(&out[oy * W + ox], acc);
26         }
27         for (; ox < W; ++ox) {
28             float acc = 0.0f;
```

```

28         for (int ky = 0; ky < K; ++ky)
29             for (int kx = 0; kx < K; ++kx)
30                 acc += in[(oy + ky) * in_stride + (ox + kx)]
31                     * ker[ky * K + kx];
32         out[oy * W + ox] = acc;
33     }
34 }

```

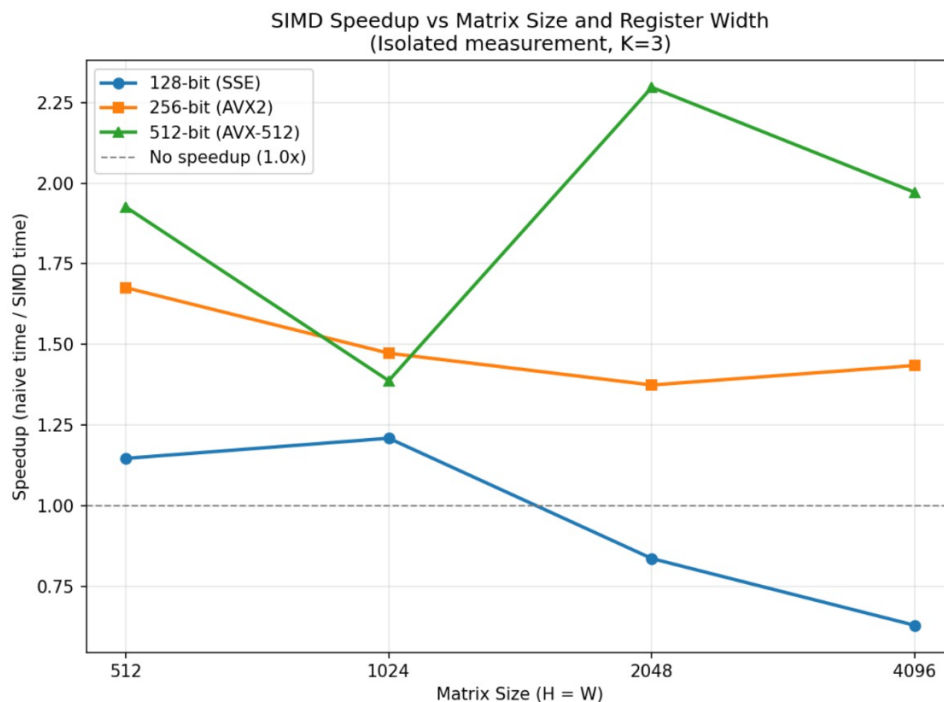
3. Instruction Count and Performance Analysis:

Matrix Size (H=W)	K	Naive Instructions	SIMD Instructions	Instruction Ratio (Naive/SIMD)	Naive Time (s)	SIMD Time (s)	Speedup (Time)
512	3	26952444	6092327	4.42	0.010615	0.004041	2.63
1024	3	102467879	18993235	5.39	0.018267	0.015150	1.21
2048	3	404496690	70553687	5.73	0.082708	0.064604	1.28
4096	3	1612546384	276709490	5.83	0.259393	0.080886	3.21

SIMD clearly outperformed the naive implementation, giving speedups between 1.21x and 3.21x across all matrix sizes. This is mainly because SIMD processes multiple data elements per instruction, cutting the total instruction count by roughly 4-6x as compared to naive. The speedup isn't perfectly smooth though. It dips a bit at mid-sized matrices (1024, 2048) before jumping to its best value (3.21x) at 4096.

4. Multi-Size Evaluation:

Matrix Size (H=W)	K	Naive Instr	Naive Time (s)	128-bit Instr	128-bit Time (s)	128-bit Speedup	256-bit Instr	256-bit Time (s)	256-bit Speedup	512-bit Instr	512-bit Time (s)	512-bit Speedup
512	3	26952444	0.010615	9658209	0.009259	1.15	6092329	0.006334	1.68	4683292	0.005510	1.93
1024	3	102467879	0.018267	33272719	0.015110	1.21	18993234	0.012403	1.47	13357129	0.013166	1.39
2048	3	404496690	0.082708	127690644	0.098953	0.84	70553687	0.060192	1.37	48009291	0.036003	2.30
4096	3	1612546384	0.259393	505282463	0.412657	0.63	276709467	0.180809	1.43	186531918	0.131573	1.97



ANSWERING QUESTIONS

1. Moving from naive to SIMD cut the instruction count drastically, roughly 4.4x to 5.8x fewer instructions, growing with matrix size (from 512 up to 4096). This happens because AVX2's 256-bit registers process 8 float values in a single instruction instead of one at a time, so each SIMD fused multiply-add does the work of 8 naive multiply-adds. The ratio improves slightly as matrix size grows because larger matrices give more full chunks of 8 to vectorize, so the leftover scalar tail (for widths not divisible by 8) matters less.

2. Yes, speedups ranged from 1.21x to 3.21x. This came from doing 8 computations per instruction and precomputing/broadcasting kernel weights once instead of per pixel.

3. We used AVX2 256-bit intrinsics:

- `_mm256_set1_ps` – to broadcast each kernel weight across all 8 lanes
- `_mm256_loadu_ps/_mm256_storeu_ps` – to load/store 8 input or output values at once, using unaligned access since memory wasn't guaranteed to be aligned
- `_mm256_setzero_ps` – to initialize the accumulator
- `_mm256_fmadd_ps` – fused multiply-add, so multiply and add happen in one instruction instead of two, saving both instructions and rounding error

1.4 Part D

Listing 5: TILING + SIMD

```
1 #include <immintrin.h>
2 #include "convolution.h"
3
4 void conv_optimized(const float* in, float* out, const float* ker
5 , int H, int W, int K) {
6     const int PW = W + 2;    // K is assumed to be 3
7
8     __m256 w0 = _mm256_set1_ps(ker[0]);
9     __m256 w1 = _mm256_set1_ps(ker[1]);
10    __m256 w2 = _mm256_set1_ps(ker[2]);
11    __m256 w3 = _mm256_set1_ps(ker[3]);
12    __m256 w4 = _mm256_set1_ps(ker[4]);
13    __m256 w5 = _mm256_set1_ps(ker[5]);
14    __m256 w6 = _mm256_set1_ps(ker[6]);
15    __m256 w7 = _mm256_set1_ps(ker[7]);
16    __m256 w8 = _mm256_set1_ps(ker[8]);
17
18    for (int y = 0; y < H; y++) {
19        const float* r0 = in + (y + 0) * PW;
20        const float* r1 = in + (y + 1) * PW;
21        const float* r2 = in + (y + 2) * PW;
22        for (int x = 0; x < W; x += 8) {
```

```

22     __m256 acc = _mm256_mul_ps(_mm256_loadu_ps(r0 + x),
23                               w0);
24     acc = _mm256_fmadd_ps(_mm256_loadu_ps(r0 + x + 1), w1
25                           , acc);
26     acc = _mm256_fmadd_ps(_mm256_loadu_ps(r0 + x + 2), w2
27                           , acc);
28     acc = _mm256_fmadd_ps(_mm256_loadu_ps(r1 + x),      w3
29                           , acc);
30     acc = _mm256_fmadd_ps(_mm256_loadu_ps(r1 + x + 1), w4
31                           , acc);
32     acc = _mm256_fmadd_ps(_mm256_loadu_ps(r1 + x + 2), w5
33                           , acc);
34     acc = _mm256_fmadd_ps(_mm256_loadu_ps(r2 + x),      w6
35                           , acc);
36     acc = _mm256_fmadd_ps(_mm256_loadu_ps(r2 + x + 1), w7
37                           , acc);
38     acc = _mm256_fmadd_ps(_mm256_loadu_ps(r2 + x + 2), w8
39                           , acc);
40     _mm256_storeu_ps(out + y * W + x, acc);
41 }
42 }
43 }

```

we got best results for unrolling + SIMD.

